

Tipos de Dato

La noción de tipo

La mayoría de los lenguajes incluye esta noción vinculada a expresiones y objetos de dato.

Los principales razones:

- Proveen un contexto implícito para muchas operaciones, de esta manera el programador no debe explicitar el contexto.
- Limitan el conjunto de operaciones que pueden realizarse en un programa semánticamente válido.

Generalidades

- Abstracción de datos en los lenguajes de programación
- Trata con las componentes de un programa que son sujeto de computación. Esta basada en las propiedades de los objetos de dato y las operaciones de dichos objetos.
 - Tipos de datos
 - Sistemas de tipos
 - Encapsulamiento y abstracción

Sistema de tipos: Conjunto de reglas que estructuran y organizan una colección de tipos.

El objetivo del sistema de tipos es lograr que los programas sean tan seguros como sea posible.

Sistemas de Tipos

Informalmente consiste de:

- Un mecanismo para definir tipos y asociarlos con ciertos constructores del lenguaje
- Un conjunto de reglas para equivalencias de tipos, compatibilidad, inferencia, etc.

Los constructores involucrados son aquellos que manejan valores o hacen referencias a objetos que tienen valores.

Chequeo de tipos

Es el proceso a través del cual se garantiza que un programa obedece las reglas de compatibilidad que el lenguaje de programación define.

Un lenguaje es “*estáticamente tipado*” si su tipado es fuerte y el chequeo puede hacerse en compilación.

En un sentido estricto muy pocos lenguajes entran en esta categoría.

Lo habitual es que realicen todas las tareas de chequeo posibles en compilación y el resto se realiza en ejecución.

Tipado dinámico

Ganó popularidad con los lenguajes de scripting.

Responde a la pregunta:

¿vale la pena chequear todo lo que podemos en tiempo de compilación?

Clasificación de tipos

Varía de lenguaje en lenguaje.

La mayoría provee tipos pre-definidos y constructores de tipo.

Entre los predefinidos se encuentran:

- Tipos numéricos
- Caracteres
- Booleanos

Constructores básicos:

- Enumerados
- Subrangos

Tipos compuestos:

- Registros
- Uniones (registros variantes)
- Arreglos
- Conjuntos
- Punteros
- Listas
- Archivos

Sistemas de Tipos

Con frecuencia la seguridad compromete la flexibilidad del lenguaje

Cual de las dos características termina siendo la más preponderante o buscada por el sistema depende de a qué tipo de desarrollos se apunte con el lenguaje de programación

Los lenguajes orientados al desarrollo de aplicaciones complejas buscan mayor seguridad y se caracterizan por tener un sistema de tipos estricto.

Los lenguajes script orientados al desarrollo rápido, buscan mayor flexibilidad de modo que se caracterizan por tener un sistema de tipos relajado

Sistemas de Tipos: Dureza del tipado

El sistema de tipos determina el **grado de dureza** respecto a una característica, cuando especifica restricciones sobre cómo se incorpora o manipula esa característica en el lenguaje.

Por ejemplo, el grado de dureza del sistema tipos con respecto a las expresiones mixtas

Tipado menos duro	Tipado más duro
<pre>A = 2 B= "2" Concatenar(A,B) {retorna "22"} Sumar(A,B) {retorna 4}</pre>	<pre>A = 2 B= "2" Concatenar(A,B) {error de tipos} Sumar(A,B) {error de tipos} Concatenar(srt(A),B) {retorna "22"} Sumar(A,int(B)) {retorna 4}</pre>

Sistemas de Tipos: Especificación

- Tipo y tiempo de chequeo
- Reglas de equivalencia y conversión
- Reglas de inferencia de tipo
- Nivel de polimorfismo del lenguaje

Para cada ítem se puede determinar el grado de dureza del sistema de tipos.

Observación importante: cada ítem puede involucrar varias características que se pueden evaluar en forma independiente.

Tipo y tiempo de chequeo

- El chequeo de tipo, consiste en verificar que el tipo de una entidad corresponda a esperado por el contexto.
- Si no coinciden, puede tratarse de un error o se aplican reglas de coerción o reglas de equivalencia.

Sistemas de Tipos: Errores de tipo

- Errores en el uso del lenguaje: Errores sintácticos o semánticos
- Errores en la aplicación: desviaciones de la aplicación con respecto a la especificación

Ej. Un acceso ilegal que al pasar inadvertido provoca un error de aplicación

Ej. Un error chequeado y notificado por el lenguaje que permite su depuración

Idealmente en el sistema de tipos se establecen restricciones que aseguran que no se van a producir errores de tipos en la aplicación, gracias a que ya han sido detectados en el lenguaje

Sistemas de Tipos: Tipo y tiempo de chequeo

Tipos de ligadura

- Tipado estático: ligaduras en compilación
- Tipado dinámico: ligaduras en ejecución, no es seguro
- Tipado seguro: no es estático, ni inseguro

Sistemas de Tipos: Lenguajes fuertemente tipados

Definición 1: Un lenguaje se dice fuertemente tipado (**type safety**) si el sistema de tipos impone restricciones que aseguran que no se producirán errores de tipo en ejecución

Definición 2: Un lenguaje se dice fuertemente tipado (**type safety**) si todos los errores de tipo se detectan

En esta concepción, la intención es evitar los errores de aplicación y son tolerados los errores del lenguaje, detectados tan pronto como sea posible

Sistemas de Tipos: Lenguajes fuertemente tipados

Un sistema de tipos puede conducir a un LP fuertemente tipado requiriendo que:

- Solo se pueden utilizar tipos predefinidos
- Todas las variables son declaradas y asociadas a un tipo específico
- Todas las operaciones son especificadas determinando con exactitud su signatura (los tipos requeridos por sus operandos y el tipo del resultado)

Haciéndose eco de la segunda definición, se puede alcanzar a la posición de fuertemente tipado, con requerimientos leves.

Es decir especificando con precisión reglas para:

- Conversión o coerción
- Equivalencia o compatibilidad

Sistemas de Tipos: Reglas de conversión

Un sistema de tipos flexible brinda reglas de conversión o coerción que permiten aceptar datos de un tipo Q en un contexto en el cual se espera un dato de tipo T

La conversión puede ser:

- Con pérdida (limitante) insegura
- Sin pérdida (expansora) segura

Sea $f: T1 \rightarrow R1$

La operación f puede ser invocada con un argumento de tipo $T2$, si el lenguaje brinda una regla que permita convertir los valores de tipo $T2$ en valores de tipo $T1$

La operación f puede ser invocada en un contexto en el se espera un valor de $R2$, si el lenguaje brinda reglas para convertir valores de tipo $R1$ en valores de tipo $R2$

Sistemas de Tipos: Reglas de conversión

En la mayoría de los lenguajes durante la asignación hay una operación implícita de dereferenciamiento:

$$x := x + 1$$

Las reglas de conversión en los lenguajes:

Fortran es un lenguaje que resuelve las conversiones de acuerdo a una jerarquía de tipos:
COMPLEX > DOUBLE PRECISION > REAL > INTEGER

Ada exige que todas las conversiones se hagan explícitamente

En Pascal las conversiones no están especificadas con precisión y dependen de la implementación

Sistemas de Tipos: Reglas de conversión en los LP

En C

```
int x,y  
real z  
x:= y + z
```

`y` se convierte a real antes de evaluar la suma y luego el resultado se convierte a `int`

El programador puede utilizar el casting para indicar explícitamente la conversión

```
x:= y + (int) z
```

C++ es similar a C

Java tiene un tratamiento diferenciado para tipos primitivos y tipo clase

Sistemas de Tipos: Reglas de equivalencia

Dos tipos de dato son equivalentes si cada uno de ellos puede aparecer en un contexto en el que opera una aparición del otro.

NO HAY CONVERSIÓN

En un lenguaje fuertemente tipado las reglas equivalencia deben quedar establecidas en el diseño del mismo.

Sistemas de Tipos: criterios de equivalencia

Equivalencia por nombre: dos variables son de tipos equivalentes si han sido declaradas en una misma instrucción o con exactamente el mismo nombre de tipo

Equivalencia por declaración: dos variables son equivalentes si conducen a la misma expresión de tipo original luego de una serie de re-declaraciones

Equivalencia por estructura: dos tipos de datos son equivalentes si están definidos por dos expresiones de tipo idénticas.

Sistemas de Tipos: equivalencia por estructura

A la hora de implementar la equivalencia por estructura es necesario plantear como resolver ciertos problemas:

1. El problema de la circularidad
2. Determinar si dos registros con diferentes nombres para sus campos son compatibles
3. Determinar si dos arreglos con la misma cardinalidad y rango son equivalentes.

Consideraciones:

- La compatibilidad por nombre es mas simple de implementar
- La compatibilidad por nombre también es mas fuerte que la compatibilidad por estructura

Sistemas de Tipos: equivalencia por estructura

type

T1: array[1..10] of int

T2: array[1..10] of int

T3: T2

var

v1, v2: array [1..10] of int

v3, v4: T1

v5: T2

v6: T2

v7: T3

Sistemas de Tipos: criterios de equivalencia en LP

En Ada

```
Type Derivado_INT is new integer range 1..100;  
Subtype Subrango_INT is integer range 1..100;
```

Adopta compatibilidad por nombre. Debido a que incorpora subtipos, define que las variables que pertenezcan a diferentes subtipos del mismo tipo base, son compatibles.

En C

Equivalencia por estructura salvo para registros y unión que tienen equivalencia por declaración

En Java

Para las clases la equivalencia esta dada por la jerarquía de herencia